Dynamic Programming:
Longest Common Subsequence (LCS)

Course Name: Analysis and Design of Algorithms
Course Code: R1UC407B
Instructor Name: Dr. Mili Dhar

Review of the key concepts of 0/1 Knapsack

1. Problem Definition:

   Given n items with weights w[i] and values p[i], and a knapsack of capacity Wc. Choose items to maximize total value without exceeding weight Wc. Each item can be chosen once (0 or 1 times) → No partial items.

2. Recursive Relation:

   if(wi > Wc)

   T(i, Wc) = T(i-1, Wc)

   else

   T(i, Wc) = max(T(i-1, Wc), T(i-1, Wc - wi) + pi)

   Base case: T(0, Wc) = 0 for all Wc.

3. Time complexity: O(n * W)

   Space complexity: O(n * W).

4. Greedy Fails: Unlike Fractional Knapsack, greedy based on value/weight ratio doesn't work.

Can you think of a way to measure how similar two strings ('ABCDEF' and 'AEBDF') are (not necessarily contiguously)?

Learning Outcome

Implement a dynamic programming solution to find the length of the LCS between two sequences.

Longest Common Subsequence (LCS)

- **Application:** comparison of two **DNA** strings
- Ex:

  X= {A B C B D A B }

  Y= {B D C A B A}
- Longest Common Subsequence:

  X = A **B** **C** **B** D **A** B

  Y = **B** D **C** A **B** **A**

Example

$X = \langle A, B, C, B, D, A, B \rangle$

$Y = \langle B, D, C, A, B, A \rangle$

$X = \langle A, B, C, B, D, A, B \rangle$

$Y = \langle B, D, C, A, B, A \rangle$

- $\langle B, C, B, A \rangle$ and $\langle B, D, A, B \rangle$ are **longest common subsequences** of X and Y (**length = 4**)

- $\langle B, C, A \rangle$, however is not a LCS of X and Y

LCS Algorithm

- First we'll find the length of LCS. Later, we'll find LCS itself.
- Define X_i, Y_j to be the prefixes of X and Y of length *i* and *j* respectively
- Define ***c[i,j]*** to be the length of LCS of X_i and Y_j
- Then the length of LCS of X and Y will be **c[m+1,n+1]**

$$c[i,j] = \begin{cases} c[i-1, j-1]+1 & \text{if } x[i] = y[j], \\ \max(c[i, j-1], c[i-1, j]) & \text{otherwise} \end{cases}$$

LCS recursive solution

$$c[i, j] = \begin{cases} c[i-1, j-1]+1 & \text{if } x[i] = y[j], \\ \max(c[i, j-1], c[i-1, j]) & \text{otherwise} \end{cases}$$

- We start with $i = j = 0$ (empty substrings of x and y)
- Since X_0 and Y_0 are empty strings, their LCS is always empty (i.e. $c[0,0] = 0$)
- LCS of empty string and any other string is empty, so for every i and j: $c[0, j] = c[i,0] = 0$

Computing the Length of the LCS

$$c[i, j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ c[i-1, j-1] + 1 & \text{if } x_i = y_j \\ \max(c[i, j-1], c[i-1, j]) & \text{if } x_i \neq y_j \end{cases}$$

Finding the LCS

A matrix b[m+1, n+1]:

- For a subproblem [i, j] it tells us what choice was made to obtain the optimal value
- **If $x_i == y_j$**

  b[i, j] = " $\nwarrow$ "

- **else, if c[i - 1, j] ≥ c[i, j-1]**

  b[i, j] = " $\uparrow$ "

- **else**

  b[i, j] = " $\leftarrow$ "

Example

$X = \langle A, B, C, B, D, A, B \rangle$

$Y = \langle B, D, C, A, B, A \rangle$

$$c[i, j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ c[i-1, j-1] + 1 & \text{if } x_i = y_j \\ \max(c[i, j-1], c[i-1, j]) & \text{if } x_i \neq y_j \end{cases}$$

If $x_i = y_j$

$b[i, j] =$ "↖"

else if $c[i - 1, j] \geq c[i, j-1]$

$b[i, j] =$ "↑"

else

$b[i, j] =$ "←"

		y_j	B	D	C	A	B	A
		0	1	2	3	4	5	6
x_i	0	0	0	0	0	0	0	0
A	1	0	↑0	↑0	↑0	↖1	←1	↖1
B	2	0	↖1	←1	←1	↑1	↖2	←2
C	3	0	↑1	↑1	↖2	←2	↑2	↑2
B	4	0	↖1	↑1	↑2	↑2	↖3	←3
D	5	0	↑1	↖2	↑2	↑2	↑3	↑3
A	6	0	↑1	↑2	↑2	↖3	↑3	↖4
B	7	0	↖1	↑2	↑2	↑3	↖4	↑4

Constructing a LCS

- Start at $b[m, n]$ and follow the arrows
- When we encounter a " ↖ " in $b[i, j] \Rightarrow x_i = y_j$ is an element of the LCS

		0	1	2	3	4	5	6
		y_j	B	D	C	A	B	A
0	x_i	0	0	0	0	0	0	0
1	A	0	↑0	↑0	↑0	↖1	←1	↖1
2	B	0	↖1	←1	←1	↑1	↖2	←2
3	C	0	↑1	↑1	↖2	←2	↑2	↑2
4	B	0	↖1	↑1	↑2	↑2	↖3	←3
5	D	0	↑1	↖2	↑2	↑2	↑3	↑3
6	A	0	↑1	↑2	↑2	↖3	↑3	↖4
7	B	0	↖1	↑2	↑2	↑3	↖4	↑4

LCS-LENGTH(X, Y, m, n)

```
for i ← 1 to m
    do c[i, 0] ← 0
for j ← 0 to n
    do c[0, j] ← 0
for i ← 1 to m
    do for j ← 1 to n
            do if x_i = y_j
                    then c[i, j] ← c[i - 1, j - 1] + 1
                         b[i, j ] ← "↖"
                    else if c[i - 1, j] ≥ c[i, j - 1]
                            then c[i, j] ← c[i - 1, j]
                                 b[i, j] ← "↑"
                            else c[i, j] ← c[i, j - 1]
                                 b[i, j] ← "←"
return c and b
```

The length of the LCS if one of the sequences is empty is zero

Case 1: $x_i = y_j$

Case 2: $x_i \neq y_j$

Running time: $\Theta(mn)$

PRINT-LCS(b, X, i, j)

Running time: $\Theta(m + n)$

```
if i = 0 or j = 0
    then return
if b[i, j] = "↖"
    then PRINT-LCS(b, X, i - 1, j - 1)
        print x_i
elseif b[i, j] = "↑"
        then PRINT-LCS(b, X, i - 1, j)
        else PRINT-LCS(b, X, i, j - 1)
```

Initial call: PRINT-LCS(b, X, length[X], length[Y])

Post Class Assesment

- You have two strings **X = BACDB** and **Y = BDCB**

  Now find the **longest common subsequence**.

Thank you!